

**Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б. Н. Ельцина»
ИРИТ-РТФ**

ЛАБОРАТОРНАЯ РАБОТА №3

Коллекции, алгоритмы и алгоритмическая сложность

ОТЧЕТ

Студент: Киприн Сергей Андреевич РИМ-250950

Екатеринбург 2026

Задачи

1. Инициализировать проект.
2. Реализовать сортировку массива. Анализ в терминах $O(n)$.
3. Реализовать алгоритм бинарного поиска в массиве. Анализ в терминах $O(n)$.
4. Решить задачу. В качестве входных данных - строка, содержащая круглые, квадратные, фигурные и угловые скобки `()[]<>{}.` Проверить, что скобки сбалансированы.

Ссылка на репозиторий Github

<https://github.com/angst-storm/js-2026>

Ход выполнения

Был реализован алгоритм быстрой сортировки. Выбирается опорный элемент из середины, остальные элементы разбиваются на «меньше» и «больше или равны», части рекурсивно сортируются и склеиваются в результат. Сложность такого алгоритма: $O(n \log n)$.

```
/**
 * Быстрая сортировка (QuickSort)
 * Сложность:  $O(n \log n)$ 
 */

export function quickSort(arr) {
  if (arr.length <= 1) {
    return arr.slice();
  }

  const pivotIndex = Math.floor(arr.length / 2);
  const pivot = arr[pivotIndex];

  const left = [];
  const right = [];
```

```

    for (let i = 0; i < arr.length; i++) {
      if (i === pivotIndex) continue;
      if (arr[i] < pivot) {
        left.push(arr[i]);
      } else {
        right.push(arr[i]);
      }
    }

    return [...quickSort(left), pivot, ...quickSort(right)];
  }
}

```

Был реализован алгоритм бинарного поиска. На каждом шаге берётся середина отсортированного массива: если элемент найден — возвращается его индекс, иначе поиск продолжается в левой или правой половине. Если элемент не найден, возвращается -1. Сложность такого алгоритма: $O(\log n)$.

```

/**
 * Бинарный поиск
 * Сложность:  $O(\log n)$ 
 * Массив должен быть отсортирован.
 */

export function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (arr[mid] === target) {
      return mid;
    }

    if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return -1;
}

```

Был реализован алгоритм проверки сбалансированности скобок $()[]\langle\rangle\{\}$. Открывающие скобки кладутся в стек, а при встрече закрывающей

проверяется, что наверху стека лежит парная открывающая. Если пары не совпали или стек в конце не пуст — последовательность некорректна. Сложность такого алгоритма: $O(n)$.

```
/**
 * Проверка сбалансированности скобок ()[]<>{}
 * Сложность: O(n)
 */

const PAIRS = {
  ')': '(',
  ']': '[',
  '>': '<',
  '}': '{',
};

const OPEN = new Set(Object.values(PAIRS));

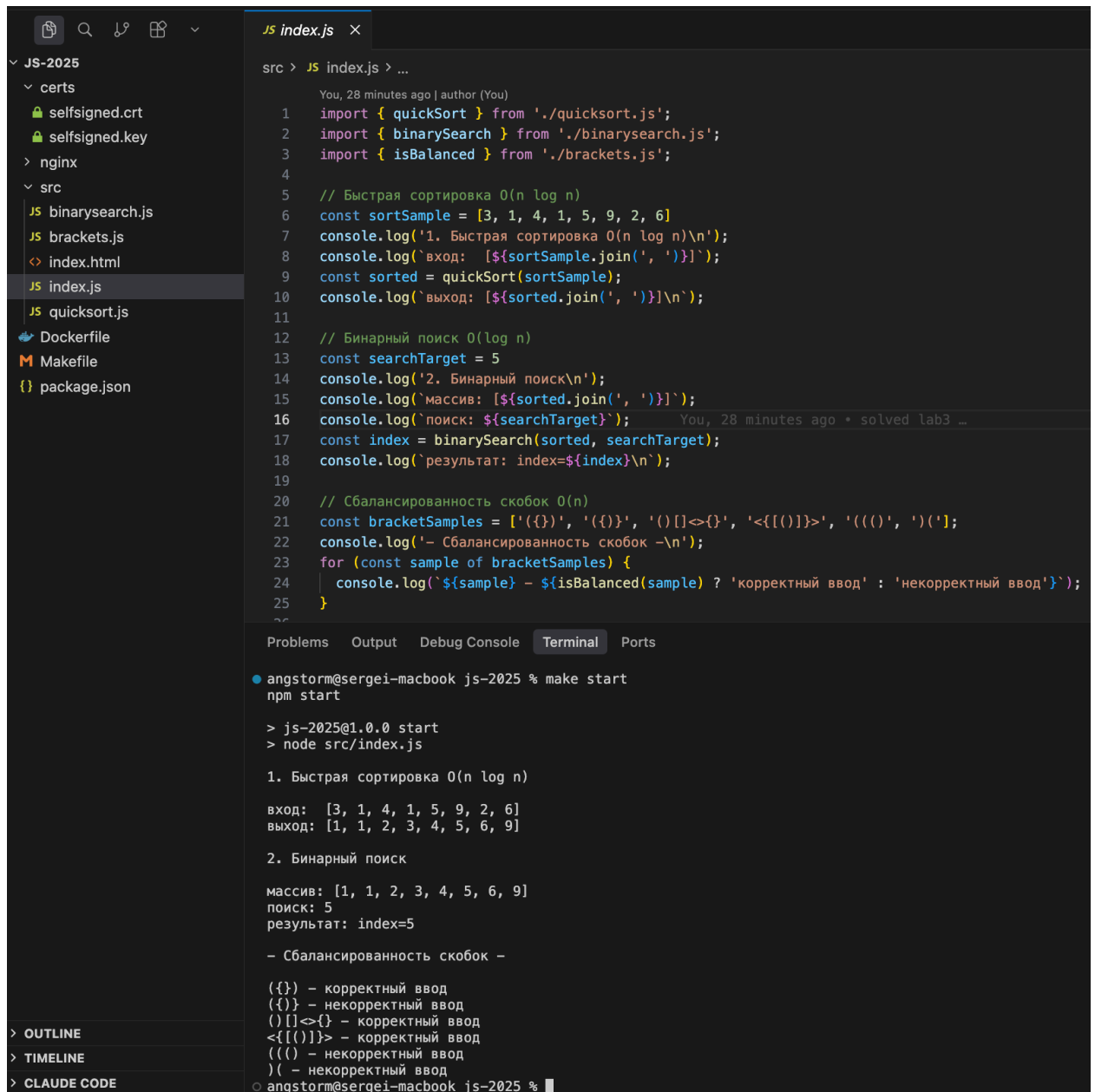
export function isBalanced(str) {
  const stack = [];

  for (const char of str) {
    if (OPEN.has(char)) {
      stack.push(char);
      continue;
    }

    if (char in PAIRS) {
      if (stack.pop() !== PAIRS[char]) {
        return false;
      }
    }
  }

  return stack.length === 0;
}
```

Был инициализирован проект и создан index.js, который проверяет работу всех алгоритмов. Ниже приведен index.js и логи работы.



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure for 'JS-2025' with files like 'selfsigned.crt', 'selfsigned.key', 'nginx', 'src', 'binarysearch.js', 'brackets.js', 'index.html', 'index.js', 'quicksort.js', 'Dockerfile', 'Makefile', and 'package.json'. The 'index.js' file is selected and its content is displayed in the editor. The code implements three algorithms: quicksort, binary search, and a bracket balancer. The terminal shows the output of running 'npm start', which executes 'node src/index.js'. The output displays the results of each algorithm: quicksort sorting an array, binary search finding an index, and a list of bracket sequences categorized as correct or incorrect.

```
JS index.js x
src > JS index.js > ...
You, 28 minutes ago | author (You)
1 import { quickSort } from './quicksort.js';
2 import { binarySearch } from './binarysearch.js';
3 import { isBalanced } from './brackets.js';
4
5 // Быстрая сортировка O(n log n)
6 const sortSample = [3, 1, 4, 1, 5, 9, 2, 6];
7 console.log('1. Быстрая сортировка O(n log n)\n');
8 console.log(`вход:  ${sortSample.join(' ')}\n`);
9 const sorted = quickSort(sortSample);
10 console.log(`выход:  ${sorted.join(' ')}\n`);
11
12 // Бинарный поиск O(log n)
13 const searchTarget = 5
14 console.log('2. Бинарный поиск\n');
15 console.log(`массив:  ${sorted.join(' ')}\n`);
16 console.log(`поиск:  ${searchTarget}`);
17 const index = binarySearch(sorted, searchTarget);
18 console.log(`результат: index=${index}\n`);
19
20 // Сбалансированность скобок O(n)
21 const bracketSamples = ['({})', '({})', '()[]<=>{}', '<{[()]>', '((((), ')((');
22 console.log('- Сбалансированность скобок -\n');
23 for (const sample of bracketSamples) {
24   console.log(`${sample} - ${isBalanced(sample)} ? 'корректный ввод' : 'некорректный ввод'`);
25 }
26
Problems Output Debug Console Terminal Ports
● angstorm@sergei-macbook js-2025 % make start
npm start

> js-2025@1.0.0 start
> node src/index.js

1. Быстрая сортировка O(n log n)

вход:  [3, 1, 4, 1, 5, 9, 2, 6]
выход: [1, 1, 2, 3, 4, 5, 6, 9]

2. Бинарный поиск

массив: [1, 1, 2, 3, 4, 5, 6, 9]
поиск:  5
результат: index=5

- Сбалансированность скобок -

({}) - корректный ввод
({}) - некорректный ввод
()[]<=>{} - корректный ввод
<{[()]> - корректный ввод
((( - некорректный ввод
)(( - некорректный ввод
○ angstorm@sergei-macbook js-2025 %
```

Вывод

В ходе работы реализованы алгоритмы быстрой сортировки, бинарного поиска и проверки сбалансированности скобок с оценкой их сложности. Полученные решения подтверждают ожидаемые оценки $O(n \log n)$, $O(\log n)$ и $O(n)$ соответственно.